

1. application.properties

src/main/resources/application.properties

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
logging.level.org.hibernate.SQL=trace
```

2. SQL – Country Table

MySQL

```
create table country(co_code varchar(2) primary key, co_name varchar(50));
insert into country values ('IN', 'India');
insert into country values ('US', 'United States of America');
insert into country values ('AU', 'Australia');
```

3. Country.java (Entity)

com/cognizant/ormlearn/model/Country.java

```
@Entity
@Table(name = "country")
public class Country {

    @Id
    @Column(name = "co_code")
    private String code;

    @Column(name = "co_name")
    private String name;

    public String getCode() { return code; }
    public void setCode(String code) { this.code = code; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}
```

4. CountryRepository.java

com/cognizant/ormlearn/repository/CountryRepository.java

```
@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
    List<Country> findByNameContaining(String keyword);
    List<Country> findByNameStartingWith(String prefix);
    List<Country> findAllOrderByNameAsc();
}
```

5. CountryService.java

com/cognizant/ormlearn/service/CountryService.java

```
@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;
```

```
@Transactional
public List<Country> getAllCountries() {
    return countryRepository.findAll();
}

@Transactional
public Country findCountryByCode(String code) {
    return countryRepository.findById(code).orElse(null);
}

@Transactional
public void addCountry(Country country) {
    countryRepository.save(country);
}

@Transactional
public List<Country> searchByName(String keyword) {
    return countryRepository.findByNameContaining(keyword);
}
}
```

6. OrmLearnApplication.java

com/cognizant/ormlearn/OrmLearnApplication.java

```
@SpringBootApplication
public class OrmLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);
    private static CountryService countryService;

    public static void main(String[] args) {
        ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);
        countryService = context.getBean(CountryService.class);
        LOGGER.info("Inside main");

        testGetAllCountries();
        testGetCountryByCode();
        testAddCountry();
        testQueryMethods();
    }

    private static void testGetAllCountries() {
        LOGGER.info("Start");
        List<Country> list = countryService.getAllCountries();
        LOGGER.debug("countries={}", list);
        LOGGER.info("End");
    }

    private static void testGetCountryByCode() {
        LOGGER.info("Start");
        Country c = countryService.findCountryByCode("IN");
        LOGGER.debug("country={}", c);
        LOGGER.info("End");
    }

    private static void testAddCountry() {
        LOGGER.info("Start");
        Country c = new Country();
        c.setCode("JP");
        c.setName("Japan");
        countryService.addCountry(c);
        LOGGER.info("Country added successfully");
        LOGGER.info("End");
    }

    private static void testQueryMethods() {
        LOGGER.info("Start");
        List<Country> list = countryService.searchByName("United");
        LOGGER.debug("Result={}", list);
        LOGGER.info("End");
    }
}
```

Output

```
INFO OrmLearnApplication Inside main

INFO OrmLearnApplication Start [testGetAllCountries]
Hibernate: select co_code, co_name from country
DEBUG OrmLearnApplication countries=[Country [code=AU, name=Australia],
```

```
Country [code=IN, name=India], Country [code=US, name=United States of America]]
INFO  OrmLearnApplication  End

INFO  OrmLearnApplication  Start [testGetCountryByCode]
Hibernate: select co_code, co_name from country where co_code=?
DEBUG OrmLearnApplication  country=Country [code=IN, name=India]
INFO  OrmLearnApplication  End

INFO  OrmLearnApplication  Start [testAddCountry]
Hibernate: insert into country (co_name, co_code) values (?, ?)
INFO  OrmLearnApplication  Country added successfully
INFO  OrmLearnApplication  End

INFO  OrmLearnApplication  Start [testQueryMethods]
Hibernate: select co_code, co_name from country where co_name like '%United%'
DEBUG OrmLearnApplication  Result=[Country [code=US, name=United States of America]]
INFO  OrmLearnApplication  End
```

7. O/R Mapping – Employee & Department

SQL

```
create table department(dept_id int primary key auto_increment, dept_name varchar(50));
create table employee(emp_id int primary key auto_increment, emp_name varchar(100),
    emp_salary double, dept_id int, foreign key(dept_id) references department(dept_id));

insert into department values(1,'Engineering'),(2,'HR');
insert into employee(emp_name,emp_salary,dept_id)
    values('Alice',75000,1),('Bob',72000,1),('Carol',65000,2);
```

Department.java

```
@Entity
@Table(name = "department")
public class Department {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "dept_id") private int id;

    @Column(name = "dept_name") private String name;

    @OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
    private List<Employee> employees;

    // getters, setters, toString
}
```

Employee.java

```
@Entity
@Table(name = "employee")
public class Employee {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "emp_id") private int id;

    @Column(name = "emp_name") private String name;
    @Column(name = "emp_salary") private double salary;

    @ManyToOne
    @JoinColumn(name = "dept_id")
    private Department department;

    // getters, setters, toString
}
```

Output

```
Hibernate: select e.emp_id, e.emp_name, e.emp_salary, d.dept_id, d.dept_name
    from employee e inner join department d on e.dept_id=d.dept_id

DEBUG employees=[
    Employee [id=1, name=Alice, salary=75000.0, dept=Engineering],
    Employee [id=2, name=Bob, salary=72000.0, dept=Engineering],
    Employee [id=3, name=Carol, salary=65000.0, dept=HR]
]
```

8. HQL & Native Queries

EmployeeRepository.java

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {

    // HQL
    @Query("SELECT e FROM Employee e WHERE e.salary > :minSalary")
    List<Employee> findBySalaryGreaterThan(@Param("minSalary") double salary);

    @Query("SELECT e FROM Employee e JOIN e.department d WHERE d.name = :deptName")
    List<Employee> findByDepartmentName(@Param("deptName") String deptName);

    // Native SQL
    @Query(value = "SELECT * FROM employee WHERE emp_salary BETWEEN :low AND :high",
            nativeQuery = true)
    List<Employee> findBySalaryRange(@Param("low") double low, @Param("high") double high);
}
```

OrmLearnApplication.java – testHQLQueries()

```
private static void testHQLQueries() {
    LOGGER.info("Start");

    List<Employee> highEarners = employeeService.getHighEarners(70000);
    LOGGER.debug("High earners={}", highEarners);

    List<Employee> engTeam = employeeService.getByDept("Engineering");
    LOGGER.debug("Engineering={}", engTeam);

    List<Employee> midRange = employeeService.getSalaryRange(65000, 76000);
    LOGGER.debug("Mid-range={}", midRange);

    LOGGER.info("End");
}
```

Output

```
INFO OrmLearnApplication Start

Hibernate: select emp_id, emp_name, emp_salary from employee where emp_salary > 70000.0
DEBUG high earners=[
    Employee [id=1, name=Alice, salary=75000.0],
    Employee [id=2, name=Bob, salary=72000.0]
]

Hibernate: select e.emp_id, e.emp_name, e.emp_salary from employee e
inner join department d on e.dept_id=d.dept_id where d.dept_name='Engineering'
DEBUG Engineering=[
    Employee [id=1, name=Alice, salary=75000.0],
    Employee [id=2, name=Bob, salary=72000.0]
]

Hibernate: SELECT * FROM employee WHERE emp_salary BETWEEN 65000.0 AND 76000.0
DEBUG mid-range=[
    Employee [id=1, name=Alice, salary=75000.0],
    Employee [id=2, name=Bob, salary=72000.0],
    Employee [id=3, name=Carol, salary=65000.0]
]

INFO OrmLearnApplication End
```